IET Information Security

Research Article

IET Journals
The Institution of Engineering and Technology

ANOVUL: Detection of logic vulnerabilities in annotated programs via data and control flow analysis

ISSN 1751-8709
Received on 3rd December 2018
Revised 23rd April 2019
Accepted on 13th January 2020
E-First on 7th February 2020
doi: 10.1049/iet-ifs.2018.5615
www.ietdl.org

Mahmoud Ghorbanzadeh[1], Hamid Reza Shahriari[1] ✉

[1]*Department of Computer Engineering, Amirkabir University of Technology, Tehran, Iran*

✉ *E-mail: shahriari@aut.ac.ir*

Abstract: Logic vulnerabilities are largely dependent on the expected functions of web applications. Their appearance depends on both application logic and related security policy which may change based on modifications in business requirements. Accordingly, there are no specific and common patterns for logic vulnerabilities moreover, a security policy is required for their detection. In this study, a vulnerability detection method is proposed to detect logic vulnerabilities via analysing the program source code. Security checks enforce some constraints in the application so that the application behaves according to the logic intended by the programmer. The main goal is to find the vulnerabilities caused by bypassing some security checks. In this method, known as annotation-based vulnerability detection approach (ANOVUL), control and data flows are analysed to detect the application logic vulnerabilities. To analyse the flows of the program, access control and authenticity labelling are used. To evaluate ANOVUL, the authors have collected a data set. This comprises of PHP applications with reported logic vulnerabilities that have common vulnerabilities and exposures (CVE) identifiers. Based on the results, a 73% detection rate was achieved in the data set. The proposed method can detect logic vulnerabilities that are not detectable using conventional methods.

1 Introduction

According to the three-layer architecture presented in [1, 2], the program is considered in three separate layers, namely the data layer, the application logic layer and the presentation layer. Each of these layers is prone to one vulnerability or the other. Generally speaking, web application vulnerabilities can be categorised into two categories of technical and logical vulnerabilities [3, 4].

Technical vulnerabilities are caused by inappropriate validation of the program inputs. In other words, input validation weakness. Hence, a hypertext transfer protocol (HTTP) request related to exploiting a technical vulnerability is an abnormal request because the web requests contain inputs that are not well-formed. In technical vulnerabilities, the attacker attempts to bypass the restrictions applied in the data and presentation layers. Hence, the vulnerabilities can be resolved by applying appropriate restrictions on the data layer and presentation layer. Cross site scripting (XSS) and structured query language (SQL) injection vulnerabilities are among the most common technical vulnerabilities. It is possible to detect XSS or SQL injection attacks through searching for patterns related to JavaScript or SQL commands, respectively.

On the other hand, application logic vulnerabilities are program-specific and depend on the application logic. In a real sense, they are weaknesses which are specific to the authorised functionality of the application logic and allow the occurrence of a threat through abusing that functionality. In a simple word, application logic vulnerabilities are weaknesses in the logic of the implemented program, such that the program exhibits a different behaviour from what it has been designed for. Due to some reasons such as the poor knowledge of the programmer or misunderstanding the design documentations, implementation of an application can be different from its design. Hence, the implemented program has indeed a different design from what was intended in the design phase. Most of the time, the attacker target is to bypass the restrictions imposed by the business rules in the logic layer of the program. In the case of abusing logic vulnerabilities, requests contain well-formed inputs, but the requests are unauthorised, according to one or more business rules.

A good practical example of technical and application logic vulnerabilities is a banking program. During the course of transferring fund from the source account to the destination one, the program may encounter SQL injection or XSS vulnerability. In such a case, the corresponding attacks are detectable through examining attack patterns in the program inputs. However, if the program has logical vulnerability, the challenge that is usually encountered is sending negative amount as the amount of the money. In this case, instead of transferring fund from the source account to the destination one, the fund will be transferred from destination account to the source account. Detection of this vulnerability depends on the business rule of the program, which requires understanding the program business rules. It is not possible to detect the attack carried out through inspecting the application inputs in the HTTP request. Also, there is no pattern that can be generally defined to detect application logic attacks.

The most common vulnerabilities of web applications are generally divided into three types. They are input validation, session management and application logic [5]. The input validation vulnerabilities are well known in web applications and they are caused by the lack of or incomplete user input validation [6]. Given the fact that HTTP cannot maintain the state, sessions are used to maintain state information between the client and server side. The session establishment vulnerabilities are known as session management vulnerabilities. Application logic vulnerabilities are specific to the intended functionality of a web application. Furthermore, in each implementation of a web application, such vulnerabilities are detected in different ways. [7].

There are some challenges in detecting logic vulnerabilities. In one hand, application logic vulnerabilities are application-specific [8], their detection requires a specification of the application logic. On the other hand, no general mechanism is available for describing the application logic. Thus, most conventional detection methods such as vulnerability scanners, and IDS systems are not able to detect logical flaws and relevant attacks [5]. Moreover, an attacker that attempts to exploit logic vulnerabilities can create well-formed inputs that are not detectable using conventional methods [9]. Thus, these vulnerabilities may open a security hole in the application for a long time. Considering the importance of detecting such vulnerabilities, this paper presents a method for detecting application logic vulnerabilities.

Since logic vulnerabilities are application-specific, they cannot be detected by methods that are based on input validation. Moreover, previous methods for detecting application logic

vulnerabilities have focused on a specific domain of logic vulnerabilities and analysed the related applications for that domain [7, 8, 10–13]. For example, a previously proposed method [8] has dealt with a subset of logic vulnerabilities, namely logic vulnerabilities in e-commerce applications. However, in this paper, we propose an annotation-based vulnerability detection approach (ANOVUL), in which application logic vulnerabilities are generally detected by control and data flow analysis. Hence in the proposed method, application logic vulnerabilities are detected regardless of the business rule of the applications.

In the ANOVUL method, the programmer uses annotations to specify the intended security properties in applications. In the source code, security checks have been used to enforce security properties. The general idea of this method is to find the paths of the program that are bypassing these security checks.

Generally speaking, static and dynamic analysis can be used to find application vulnerabilities. In a static analysis, vulnerabilities are detected without executing the application. Static analysis methods show more false positives compared to the dynamic analysis methods [14, 15]. In a dynamic analysis, the application is executed using run-time values and therefore, vulnerabilities can be detected in the part of the source code that is actually executed.

In ANOVUL, we have used static analysis due to its advantages (such as examining all possible execution paths, not just those that is actually executed). Furthermore, in order to reduce the false positive in static analysis, the ANOVUL method takes the source code that has been annotated by the programmer, as input. An annotation is a comment attached to a particular section of a source code that is typically ignored once the code is executed or compiled. Using annotations, a programmer can specify the security properties in the program source code. Annotations may be associated with some mistakes by itself, but it should be noted that the purpose of the proposed method is to identify the cases that are in contradiction with the intention of the programmer. In this approach, if there is an execution path that does not comply with the intended security check applied by the programmer, it will be detected.

In the proposed method, the vulnerability in the application logic will be reported in case a path in the control flow graph (CFG) found that one or more security check is bypassed. Each CFG includes multiple nodes and edges (in the form of finite sets). Each branch in the CFG is regarded as a check. In the nodes that are associated with branches, the rest of the application is executed based on examination of the check that has been applied by the programmer. For example, assume that a developer wants to implement a part of the program in which, if a user has administrator privilege, he can access a data object; otherwise his access is prevented. For this purpose, the programmer implements a security check to examine the access level of the user. Since the number of paths in a CFG is generally infinite, there may exist an application execution path in which a user without administrator privilege can manipulate the data object. Therefore in the proposed method, annotation is used on nodes of the graph (as finite sets) to allow automatic analysis of paths in the program. In this method, both control and data flow analysis are used to detect logic vulnerabilities caused by bypassing one or more security checks.

The rest of this paper includes the following sections: Section 2 deals with the logic vulnerabilities in web applications. In Section 3, the related studies on the detection of logic vulnerabilities in web applications are reviewed. The proposed method for detection of logic vulnerabilities is described in Section 4. The evaluation results of the method are presented in Section 5. Finally, Section 6 concludes the paper.

2 Application logic vulnerabilities

Software security is usually represented as a chain, all rings of which must be secure and indestructible to achieve a secure application. If one ring of the chain is weak, the whole chain is destructible and it is possible to penetrate the application. In this paper, we attempt to point out that even if all the rings of this chain are secure and impenetrable, there is still a possibility to penetrate into the application. Here, we introduce application logic vulnerabilities which occur due to weaknesses in the application logic.

In order to introduce the application logic vulnerabilities, we extend the chain example such that if the most suitable lock and the strongest chain are used to lock a bike to the strongest metallic rod, in case there is a logical weakness in the locking process, there is still the possibility of stealing the bike. For example, if the height of the metallic bar is short, it is possible to steal a bike by lifting it up from the ground and pulling out the chain lock from the bar.

There are different types of application logic vulnerabilities, based on the functions of each application. A common type of application logic vulnerabilities is the access control vulnerability which allows unauthorised access of an attacker to sensitive information and operations [5, 10, 16]. To verify user authorisation, developers perform some security checks before the user can access the sensitive information and operations. Since it is difficult to predict all execution paths leading to sensitive operations, there exists the possibility of missing security checks on certain paths of the application, which allows an attacker to access sensitive operations.

Workflow violation is another type of such vulnerabilities that allows attackers to violate the expected steps of a business workflow. If it is possible to bypass one or more steps in the workflow, the program is vulnerable. For example, a web application has workflow violation vulnerability, if an attacker can bypass tax steps [5, 10, 16].

Application logic vulnerabilities can occur as a result of violation of data flow and workflow [17]. For instance, in the case of tampering with the data flow, vulnerability can occur in a web application during transactions between the user, vendor and bank. There is a possibility of changing the vendor account. Hence, while there is no violation in the workflow and it works as expected, the data flow has been manipulated.

Forceful browsing is another vulnerability in which an attacker can guess a link to go directly to a hidden link, so as to access sensitive information. If a web application has parameter tampering vulnerability, an attacker can tamper web application input values within the HTTP requests [5].

A web application is divided into two sides: server and client sides. Since the calculations on the client side can be tampered by malicious users, it is necessary to enforce security checks on the server side, as well [5, 18, 19]. The lack of validation or validation inconsistency on the server and client sides may allow an attacker to violate the security policy. Another type of application logic vulnerability is inconsistency between client and server side security checks.

In a previous study [5], access control vulnerabilities, forceful browsing, execution after redirect (EAR) and violation of the workflow have been proposed as application logic vulnerabilities. Furthermore, missing function level access control, insecure direct object reference (IDOR), unvalidated redirects and forwards vulnerabilities that have been listed among the top ten vulnerabilities (in the OWASP 2013 [20]), are related to application logic vulnerabilities [5]. In a previous work [21], the access control vulnerabilities and the reason for their occurrence have been reviewed.

Application logic vulnerabilities mainly depend on the expected functions of applications [7]. Unlike technical vulnerabilities, the application logic vulnerabilities lack a constant pattern. They are application-specific vulnerabilities, such that there may exist a vulnerability in a program due to its business rule, while another program with a different business rule is not associated with this vulnerability. For example, the use of a module that performs the calculator operation in an online store results in vulnerability due to accepting negative inputs when summing two values. In this way, an attacker can change the price of some products into negative values before calculating the total price, so that the total sum is a positive value, smaller than the actual price. The reason behind this vulnerability is accepting negative values when calculating the total shopping basket price. Such calculator module in an accounting-related program has no vulnerability associated with the acceptance of negative values because in this kind of calculation, profit and loss are important and therefore, the

```
1  #No XSS
2    assert (output.trust?) unless (Prin.receiver==Lattice.Bot)
3  #Secrecy
4    assert (Lattice.leq (output.secrecy?, Prin.receiver))
5  #No CSRF
6    assert (Lattice.leq (Prin.receiver, Prin.sender)) if params[:post]
7    assert params[:post] if (Session.modified? || Db.modified?)
8  #Authentication
9    assert (Lattice.leq (session[Prin.Id], Prin.receiver))
10   assert (Lattice.leq (session[Prin.Id], Prin.sender)) if params[:post]
```

Fig. 1 *Specification of common security properties [23]*

program must accept both positive and negative values. Indeed, accepting negative values in the calculations is a weakness that is not solely related to the calculator module, and its use in applications with different business rules may cause such a vulnerability.

Detection of application logic vulnerabilities depends on the expected functions of a web application [7]. The exploitation of logic vulnerabilities occurs based on the business rule of the program, usually depending on the program workflow, which requires sending multiple HTTP requests. Application logic vulnerabilities can occur due to the possibility of abusing an authorised function of the program. In this way, it is difficult to detect these vulnerabilities in a fully automatic manner, considering that the requests are well-formed. In technical vulnerabilities, which are detectable due to the use of a specific pattern of malicious data in the inputs, one can perform such data to identify the weaknesses associated with the program input validation. Since the application logic vulnerability is specific to an application, it is not possible to create such an attack pattern and use it generally and for any web application. Furthermore, exploiting program logic vulnerabilities requires less technical knowledge, compared to other vulnerabilities. Hence, they can be easily accessed by the general public. The difficulty in detecting these attacks and their easy utilisation highlights the importance of identifying these vulnerabilities.

Attacks that exploit application logic vulnerabilities are known as state violation or logic attacks [5]. It is hard to detect logic attacks since they consist of well-formed input values and are syntactically considered as valid web requests. Identifying the malicious intent of such requests requires a good knowledge of the application logic. Therefore, it is very important to have an accurate specification of the application logic. In the proposed method, the programmer uses annotations to specify the intended security properties in applications.

3 Related work

In a previous study, a detailed survey on vulnerability detection methods has been presented [15]. Moreover, a good survey of the most common software vulnerabilities has been presented in [22]. As we have focused on logical vulnerability detection, the most related works, especially those using the annotation approach will be reviewed here.

Using annotations, a programmer can specify the security properties in the program source code. An annotation is a form of metadata that can be used to detect vulnerabilities. Rubyx [23] is a symbolic executor that analyse Ruby-on-Rails web applications for security vulnerabilities. Rubyx makes it possible to define security properties and then, it will automatically verify the applied properties. Rubyx can detect vulnerabilities related to insufficient authentication, cross site request forgery (CSRF) and access control bypass depending on the defined security properties. In this method, symbolic execution is used to analyse possible executable paths of an application. Specification of common security properties has been presented in Fig. 1.

Fig. 1 shows the common security properties which must be held in the program. For example, line 2 defines a condition in the string class to detect XSS vulnerabilities. During the symbolic execution, the strings that represent the user input are initialised as untrusted. Using Ruby-on-Rails sanitisation mechanisms, untrusted strings change to trusted. Based on the specification of *NoXSS*, as shown on line 2 of Fig. 1, outputs must be trusted unless the output is received by an attacker.

GuardRails [24] deals with security policies defined based on annotation, and then builds a version of a given application in which the security policies are automatically applied. In this method, security policies are directly attached to the data objects (the corresponding class) and automatically applied to all classes throughout the application.

GuardRails changes the instructions of the web application, which are annotated, such that the obtained instructions are secured. A programmer defines the desired policies using the annotation added to data model declarations. GuardRails produces necessary instructions for applying the described policies automatically, and adds them to the application during compiling. Access control policies are defined as security policies that determine whether a particular person has the right to perform an action on some desired object.

To specify access control policies, developers provide access control annotations of the form is as follows:

$$@\langle policy-type\rangle, \langle target\rangle, \langle mediator\rangle, \langle handler\rangle \quad (1)$$

where *policy-type* can accept one of the values *read*, *edit*, *append*, *create* and *destroy*. The *target* refers to the related object. The *mediator* determines if the operation is allowed or not. Ifan operation is not allowed, the *handler* function is called.

ASIDE [25] mitigates access control vulnerabilities using annotation in the program source code. In this method, the developer annotates a program source code according to the intended access control. Vulnerability detection can be achieved without undesirable impacts on program development.

Since logic vulnerabilities are application-specific, they cannot be detected by methods that are based on input validation. Previous methods for detecting application logic vulnerabilities have focused on a specific domain of logic vulnerabilities and analysed the related applications for that domain. For example, a previously proposed method [8] has dealt with a subset of general logic vulnerabilities, namely logic vulnerabilities in e-commerce applications.

Table 1 shows a comparison of the proposed method to the traditional methods. Because of the data and control flow analysis in the ANOVUL method, logic vulnerabilities that occur as a result of a data flow or control flow violation can be detected.

4 Annotation-based vulnerability detection method

In this section, a detailed description of the annotation-based vulnerability detection method (ANOVUL) is presented. Thereafter, we will have a glance to the approach and then the steps will be explained. In the end, some case study examples are provided to clarify the approach.

4.1 Overall approach

Here, the overall ANOVUL approach is described to set the ground for detecting vulnerabilities due to bypassing one or more security checks. In ANOVUL, the programmer uses annotations to specify the intended security properties in applications. In the source code, security checks have been used to enforce security properties. An annotation is a comment attached to a particular section of a source code that is typically ignored once the code is executed or compiled.

Security checks enforce some constraints in application execution path so that the application behaves according to the logic intended by the programmer. The general idea of the ANOVUL method is that it finds the paths that bypass these security checks. To analyse the paths, we make use of the CFG, in which nodes and edges are equal to the basic blocks and branches between them, respectively.

Each basic block in CFG contains a set of instructions which are executed from the beginning to the end of the block without any branch and there are no jumps in or out of the middle of a

Table 1 Comparison of the ANOVUL method with related methods

Detection method	Type of analysis				Type of vulnerability					
Static	Static	Dynamic	Monitoring	Formal method	CSRF[a]	AC[b]	Auth[c]	WFB[d]	PM[e]	DFV[f]
MiMoSA [10]	✓				✗	✓	✓	✓	✗	✗
Swaddler [16]	✓				✗	✓	✓	✓	✗	✗
Sun *et al.* [26]	✓				✗	✓	✓	✓	✗	✗
ViewPoints [27]	✓				✗	✗	✗	✗	✓	✗
NoTamper [28]		✓			✗	✗	✗	✗	✓	✗
Waler [7]	✓				✗	✓	✓	✓	✓	✗
LogicScope [29]		✓			✗	✗	✗	✓	✓	✗
FixMeUp [30]	✓				✗	✓	✓	✗	✗	✗
WAPTEC [18]	✓				✗	✗	✗	✗	✓	✗
MACE [31]		✓			✗	✓	✓	✗	✗	✗
TamperProof [32]		✓			✗	✗	✗	✓	✓	✗
ASIDE [25]	✓				✗	✓	✓	✗	✗	✗
GuardRails [24]			✓		✗	✓	✓	✗	✗	✗
Rubyx [23]				✓	✓	✓	✓	✗	✗	✗
Rocchetto *et al.* [33]				✓	✓	✗	✗	✗	✗	✗
FlowFox [34]				✓	✗	✗	✗	✗	✓	✗
Akhawe *et al.* [35]				✓	✗	✗	✗	✗	✓	✗
JSFlow [36]		✓			✗	✗	✗	✗	✓	✗
ANOVUL	✓				✓	✓	✓	✓	✓	✓

[a]CSRF: Cross site request forgery, [b]AC: Access Control bypass, [c]Auth: Authentication bypass, [d]WFB: Workflow bypass, [e]PM: Parameter manipulation, [f]DFV: Data flow violation.

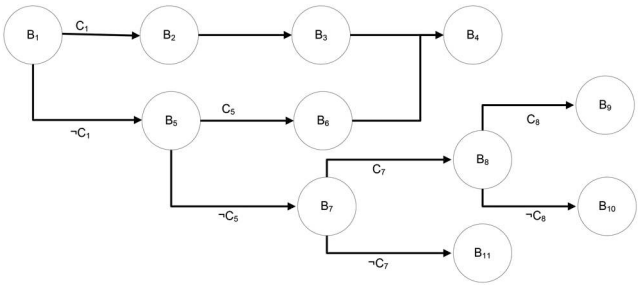

Fig. 2 *Example for a part of a CFG*

block. Generally, we can assume that operations in instructions are *read* from an *object* (e.g. memory, file, and database) or *write* to it. This abstraction would not result in the loss of required information.

After the execution of the last instruction in a basic block, the control flow is transferred to another basic block according to the corresponding branch. Each branch in the CFG is regarded as a check. A subset of these checks is related to the application logic.

Now, we define the CFG as a directed graph, as the following:

$$\begin{aligned} \mathrm{CFG} &= (N, E) \\ N &= B_{\mathrm{I}} \cup B_{\mathrm{En}} \cup B_{\mathrm{Ex}} \end{aligned} \tag{2}$$

where N is the set of nodes, including intermediate basic blocks (B_{I}), exit basic blocks (B_{Ex}), and entry basic blocks (B_{En}). E represents the set of edges between nodes. If $(B_i, B_j) \in E$, then B_i may be the predecessor of B_j in the program execution flow, according to the corresponding branch.

For a $\mathrm{CFG} = (N, E)$, an execution path is defined as $P = B_1, B_2, \ldots, B_n$, where:

$$\begin{aligned} &B_1 \in B_{\mathrm{En}} \\ &B_n \in B_{\mathrm{Ex}} \\ \forall B_i \in P, \quad &1 < i \leq n \,\big|\, (B_{(i-1)}, B_i) \in E \end{aligned} \tag{3}$$

The set of nodes and edges in a CFG is finite, but the execution paths are generally considered infinite with respect to executions of application with different inputs. It is worth noting that since cycles do not affect labelling, they are not regarded as in our method.

Fig. 2 shows a sample part of a CFG. A subset of intermediate basic blocks (B_{I}) contains the checks (i.e. branches in the CFG). These basic blocks are shown as B_{ch}. For example in Fig. 2, $B_{\mathrm{ch}} = \{B_1, B_5, B_7, B_8\}$. In Fig. 2, C_1, C_5, C_7, and C_8 constraints are verified in the checks pertaining to B_1, B_5, B_7, and B_8, respectively. A subset of these checks is related to the application logic.

In Fig. 2, constraint Cst_1 should be satisfied for execution of B_9. Assuming that C_1 and C_8 constraints are related to the application logic, the corresponding checks are regarded as the application logic checks. In order to detect logic vulnerability related to basic block B_9, the ANOVUL method evaluates whether there is an execution path on which the constraint Cst_2 is not satisfied.

In the ANOVUL method, the programmer uses annotations to specify the intended security properties in applications. In the source code, security checks have been used to enforce security properties. The general idea of this method is that it finds the paths of the program that bypassing these security checks

$$\begin{aligned} Cst_1 &: (\neg C_1) \wedge (\neg C_5) \wedge (C_7) \wedge (C_8) \\ Cst_2 &: (\neg C_1) \wedge (C_8) \end{aligned} \tag{4}$$

In ANOVUL, both control and data flows are analysed statically to detect logic vulnerabilities caused by bypassing one or more security checks. In this method, the vulnerability in the application logic will be reported in case a path in the CFG found that one or more security check is bypassed. In the proposed method, if there is an execution path that does not comply with the security check applied by the programmer, it will be detected (Fig. 3).

Algorithm 1 shows the pseudocode of the ANOVUL method. In each pass, the algorithm first selects an execution path and then calls the labelling procedure to label the operations and objects in each basic block. The procedure is described in Section 4.2. In ANOVUL, the programmer provided annotations is regarded as the correct application logic. As explained in Section 4.3, check_for_inconsistency function is called to find inconsistencies in the source code. The algorithm continues until all paths are analysed.

4.2 Labelling method

In the ANOVUL method, control and data flows are analysed so as to detect the application logic vulnerabilities, using access control (AC) and authenticity (AU) labels, respectively. According to

```
Require: Annotations, CFG, DFG
  ▷ Initilize the object labels as the lowest level
  for all object o do
      L_AU(o) ← ∅
  end for
  Paths ← all simple execution paths in CFG
  for all p ∈ Paths do
  ▷ Check all simple execution paths in CFG
      for all basic block b ∈ p do
          LABELING(b, Annotations)
          if CHECK_FOR_INCONSISTENCY(b) then
              print "vulnerability found in basic block b"
              break
          end if
      end for
  end for
```

Fig. 3 *Algorithm 1: The pseudocode of the ANOVUL vulnerability detection*

Table 2 Method of labelling

Type of label	security checks	Entities		Operations	
		subjects	objects	read	write
AC[a]	A[c]	A	A	V[d]	V
AU[b]	A	A	V	A	V

[a]AC: Access Control, [b]AU: Authenticity.

[c]A: Determined by user annotations.

[d]V: Determined by the ANOVUL method.

```
procedure LABELING(b, Annotations)
    ▷ Labeling basic block b
    if b ∈ B_En then
        L_AC(b) ← ∅
    else
        if C_prd(b) = ∅ then
            L_AC(b) ← L_AC(prd(b))
        else
            L_AC(b) ← MAX(L_AC(prd(b)), Level(C_prd(b)))
        end if
    end if
    L_AU(b) ← L_AU(subject(b))
    ▷ Labeling operations and objects
    for all read operation r ∈ b do
        L_AC(r) ← L_AC(b)
    end for
    for all write operation w ∈ b that writes in object o do
        L_AC(w) ← L_AC(b)
        L_AU(o) ← MIN(L_AU(b), L_AU(o))
    end for
end procedure
```

Fig. 4 *Algorithm 2: The pseudocode of the labelling objects and operations*

Table 2, the programmer determines some labels through annotations, whereas some others are determined by ANOVUL.

Algorithm 2 (Fig. 4) shows the pseudocode of the labelling method. The access control labels of *read* and *write* operations and the authenticity labels of *write* operations and objects are determined by execution of labelling procedure. In this algorithm, *prd*(*b*) returns the first predecessor of basic block *b* in the path.

In the *AC* labelling, the programmer uses annotations to determine the labels of *security checks*, *objects*, and *subjects*. The *AC* label of a security check indicates at what level of access control the rest of an application path is executed. The *AC* label of an object shows the confidentiality level of that object. The *AC* label of each *read* or *write* operation is determined by the ANOVUL method.

According to the security policy, each security check has an access control (*AC*) label. If the constraint of a security check (i.e. branch condition) is C_i, the corresponding *AC* label is indicated as *Level*(C_i) which is determined by verifying C_i. For instance, a security check that controls the administrator access level (to determine whether the current user is authorised for the administrator access or not) is followed by basic blocks with administrator or non-administrator access levels. In this example, *Level*(C_i) returns *A* or ∅ label, respectively.

According to Table 2, the *AC* labels of *read* and *write* operations are determined by the ANOVUL method. In this method, the *AC* labels of *read* and *write* operations pertaining to each basic block are specified with respect to the application execution paths. The *AC* label of each B_{En} is regarded as the lowest access level (∅). According to Algorithm 2, for each execution path $P = \langle B_{\text{En}}, \ldots, B_i, B_j, \ldots, B_{\text{Ex}} \rangle$, the *AC* labels of B_j basic block $(prd(B_j) = B_i)$ can be determined as the following:

$$L_{AC}(B_{\text{En}}) = \varnothing$$
$$L_{AC}(B_j) = \begin{cases} L_{AC}(B_i) & \text{if } C_i = \varnothing \\ \text{MAX}(L_{AC}(B_i), Level(C_i)) & \text{if } C_i \neq \varnothing \end{cases} \quad (5)$$

Given the annotation and depending on the branch condition, the *Level*() function returns the *AC* label of the related security check

$$Level(C_i) = \begin{cases} L_{AC}(C_i) & \text{if } C_i \text{ is satisfied on the path} \\ \neg L_{AC}(C_i) & \text{if } C_i \text{ is not satisfied on the path} \end{cases} \quad (6)$$

In the *AC* labelling method, we have the following two cases:

- B_i *lacks* C_i*:* the access control label of B_j is equal to the access control label of B_i. For instance, in Fig. 2, the access control label of B_4 is equal to the *AC* label of B_3 or B_6 with respect to the followed branch.
- B_i *has* C_i*:* the access control label of B_i is equal to the maximum access control label of B_i and *Level*(C_i). For instance, in Fig. 2, the access control label of B_6 is equal to MAX($L_{AC}(B_5)$, *Level*(C_5)), and the result of *Level*(C_5) for B_6 is equal to $L_{AC}(C_5)$.

To analyse data flow, an authenticity label is considered for each data object. This authenticity label indicates the authenticity level of the data provider. Each *subject* has an authenticity label. Considering what subject executes a B_i, each instruction in this

```
function CHECK_FOR_INCONSISTENCY(b)
    for all read operation r ∈ b that reads from object o do
        if L_AC(r) < L_AC(o) then
            return True
        end if
        if L_AU(o) < L_AU(r) then
            return True
        end if
    end for
    for all write operation w ∈ b that writes in object o do
        if L_AC(w) < L_AC(o) then
            return True
        end if
    end for
    return False
end function
```

Fig. 5 *Algorithm 3: The pseudocode of the vulnerability detection method*

```
1. <?php
2.    if(IsNotAdmin($_SESSION['Admin'])) //Anno-
       tation1;
3.      { header("location: /login.php"); } //A-
          nnotation2;
4.    AddUser(); //Annotation3;
5. ?>

/*
<Annotation1>if IsNotAdmin() then Level(C2)=∅ else Level(C2)=
A </Annotation1>
<Annotation2>L_AC(login.php)= ∅</Annotation2>
<Annotation3>L_AC(Database.User_Table)= A</Annotation3>
*/
```

Fig. 6 *Example 1: An example of execution after redirect vulnerability. The annotations were added as comments to the example*

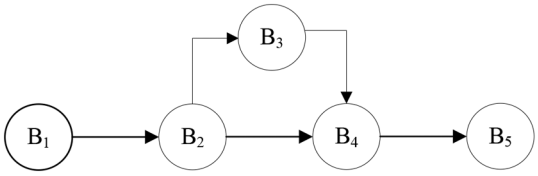

Fig. 7 *CFG for Example 1. This is an example of EAR vulnerability*

basic block has an authenticity label which is applied to data object to change the data object authenticity label.

In the *AU* labelling, the programmer uses annotations to determine the labels of *subjects*, *security checks*, and *read* operations (Table 2). The *AU* label of a security check indicates at what level of data authenticity, the rest of an application path is executed. The *AU* label of a *read* operation shows the allowed data flow for the respective *object*.

According to Table 2, the *AU* labels of *write* operation and *objects* are determined by the ANOVUL method. If the *write* operation in B_i is performed on o_i, its authenticity label is defined as below. $L'_{AU}(o_i)$ is equal to the authenticity label of o_i before executing B_i

$$L_{AU}(o_i) = \text{MIN}(L_{AU}(B_i), L'_{AU}(o_i)) \tag{7}$$

The main idea of this method is that the number of *reads*, *writes*, *objects*, and *security checks* in an application is in the form of a finite set. It is possible to determine labels for them; however, the number of possible paths of the application is generally regarded as an infinite set.

4.3 Detecting vulnerabilities

In the proposed method, both control and data flow analysis are used to detect logic vulnerabilities caused by bypassing one or more security checks. After investigating the *AC* labels of *objects* and *security checks* resulting from the annotations as well as the *AC* labels of *read* and *write* operations determined by ANOVUL, control flow violations can be detected. *AU* labels are similarly used to detect data flow violations.

In each *read* operation, the *object* access control label should be less than or equal to the *read* operation label in the execution path. In each *write* operation, it is only authorised to write into the *object* with an access control label less than or equal to the *write* operation label. Therefore, the data object access control label should be less than or equal to the *write* operation label in the execution path.

Regardless of the subject executing the *read* operation, each *read* has an authenticity label which is determined with respect to the annotations. Execution of a *read* operation is authorised when the authenticity label of the relevant data object is greater than or equal to the authenticity label of the *read* operation.

Algorithm 3 (Fig. 5) shows the pseudocode of the vulnerability detection method. For each basic block in a path, the check_for_inconsistency function is called to detect vulnerability.

4.4 Case studies

Common types of access control vulnerabilities detected by the ANOVUL method include EAR, IDOR, forced browsing, and bypassing user interface level access control. In this section, the ANOVUL method is described based on some examples.

EAR vulnerability emerges when a programmer intends to prevent execution of instructions in a path and redirect the user to another page. However, during implementation, while the user is redirected to another page, the application continues executing the rest of the instructions in the previous page. In fact, the application has vulnerabilities in the redirection procedure which leads to the execution of unwanted instructions.

In Example 1 (Fig. 6), the application has been developed in a way that only an administrator can add a new user, and an unauthorised user should be redirected to the login.php page. In PHP, header function redirects users, but it does not terminate the execution of instructions on that page.

The annotations were added to Example 1. The CFG of this example can be seen in Fig. 7. In this example, $B_1 \in B_{\text{En}}$ and $B_5 \in B_{\text{Ex}}$. Moreover, *Path1* and *Path2* are possible paths in this graph:

$$\begin{aligned} Path1 &= \langle B_1, B_2, B_3, B_4, B_5 \rangle \\ Path2 &= \langle B_1, B_2, B_4, B_5 \rangle \end{aligned} \tag{8}$$

In Example 1, the session variable, *login.php*, and the database are regarded as objects o_1, o_2, and o_3, respectively. It is assumed that there are two levels in the application: *administrator* and *guest*. In this example, instructions are simplified to *read* and *write* operations. Therefore, the instructions on lines 2, 3, and 4 are regarded as $read1(o_1)$, $write1(o_2)$, and $write2(o_3)$, respectively. Therefore, in this example we have:

$$\begin{aligned} &o_1 = \$\_SESSION[`Admin']), \quad o_2 = login\,.\,php, \\ &o_3 = Database\,.\,User\_Table, \\ &C_1 = C_3 = C_4 = C_5 = \varnothing, \quad C_2 = Is\,Not\,Admin(o_1), \\ &read1 = IsNotAdmin(), \quad write1 = header(), \\ &write2 = AddUser(), \\ &B_1 \in B_{En}, \quad B_5 \in B_{Ex}, \\ &B_2, B_3, B_4 \in B_{B_I}, \\ &B_2 = \{C_2 = read1(o_1);\ if(C_2)\}, \quad B_3 = \{write1(o_2);\}, \\ &B_4 = \{write2(o_3);\} \end{aligned} \tag{9}$$

Considering Example 1, the following information is provided as annotations for the ANOVUL method. Thus, access to o_1 and o_2 can be at the guest level (the lowest possible level). Access to o_3 requires administrator privilege, hence the security check $C_2 = IsNotAdmin(\$\_SESSION['Admin'])$ verifies that

Table 3 Results of labelling by the ANOVUL method in Example 1

AC labelling	*Path*1	*Path*2
operation *read*1	∅	∅
operation *write*1	∅	N/A
operation *write*2	∅	A

unauthorised access to o_3 is blocked. If this condition is met, the Level() function returns the guest level ($Level(C_2) = L_{AC}(C_2) = \varnothing$); otherwise, the administrator level is returned ($Level(C_2) = \neg L_{AC}(C_2) = A$). Equation (10) shows the results of labelling based on annotations in Example 1.

$$\begin{aligned} &L_{AC}(o_1) = \varnothing \\ &L_{AC}(o_2) = \varnothing \\ &L_{AC}(o_3) = A \\ &Level(C_2) = \begin{cases} L_{AC}(C_2) = \varnothing & if \;\; C_2 \, is \, satisfied \\ \neg L_{AC}(C_2) = A & if \;\; C_2 \, is \, not \, satisfied \end{cases} \end{aligned} \tag{10}$$

Algorithm 2 provides details of the labelling procedure. In the following, labelling is performed according to this algorithm. In *Path*1, the value of $L_{AC}(B_1)$, i.e. the access control label corresponding to B_1, is equal to ∅

$$B_1 \in B_{\text{En}} \Rightarrow L_{AC}(B_1) = \varnothing \tag{11}$$

In the execution of B_2, since $C_1 = \varnothing$, we will have $L_{AC}(B_2) = L_{AC}(B_1)$. Therefore, $L_{AC}(B_2) = \varnothing$ in *Path*1

$$\begin{aligned} &(p = Path1) \Rightarrow prd(B_2) = B_1 \\ &\left(C_{prd(B_2)} = C_1 = \varnothing\right) \Rightarrow \\ &(L_{AC}(B_2) = L_{AC}(prd(B_2)) = L_{AC}(B_1) = \varnothing) \\ &read1(o_1) \in B_2 \Rightarrow (L_{AC}(read1) = L_{AC}(B_2) = \varnothing) \end{aligned} \tag{12}$$

Based on Algorithm 2, $L_{AC}(read1) = \varnothing$. Furthermore, according to (10), the access control label of object o_1 is ∅. In basic block B_2, *read1* has access to an object with a ∅ access control label. According to Algorithm 3, reading object o_1 with an *AC* label less than or equal to the *AC* label of operation *read1* is authorised

$$\begin{aligned} &L_{AC}(o_1) = \varnothing \\ &L_{AC}(read1) = \varnothing \\ &(L_{AC}(read1) \not< L_{AC}(o_1)) \Rightarrow (read1(o_1) \; is \; authorised) \end{aligned} \tag{13}$$

In *Path*1, the value of $L_{AC}(B_3)$, i.e. the access control label corresponding to B_3, is equal to $\text{MAX}(L_{AC}(B_2), Level(C_2))$ which is ∅. Therefore, $L_{AC}(B_3) = \varnothing$ in *Path*1

$$\begin{aligned} &(p = Path1) \Rightarrow ((prd(B_3) = B_2) \wedge (C_2 \text{ is satisfied})) \\ &(C_2 \text{ is satisfied}) \Rightarrow (Level(C_2) = L_{AC}(C_2) = \varnothing) \\ &\left(C_{prd(B_3)} = C_2 \neq \varnothing\right) \Rightarrow (L_{AC}(B_3) = \text{MAX}(L_{AC}(prd(B_3), \\ &Level\left(C_{prd(B_3)}\right) = \text{MAX}(L_{AC}(B_2), Level(C_2)) \\ &\text{MAX}(\varnothing, \varnothing) = \varnothing) \\ &write1(o_2) \in B_3 \Rightarrow (L_{AC}(write1) = L_{AC}(B_3) = \varnothing) \end{aligned} \tag{14}$$

In operation *write1*, writing in data object o_2 with an *AC* label less than or equal to the *AC* label of operation *write1* is authorised

$$\begin{aligned} &L_{AC}(o_2) = \varnothing \\ &L_{AC}(write1) = \varnothing \\ &(L_{AC}(write1) \not< L_{AC}(o_2)) \Rightarrow (write1(o_2) \; is \; authorised) \end{aligned} \tag{15}$$

In the execution of B_4 in *Path*1, $C_3 = \varnothing$, therefore $L_{AC}(B_4) = L_{AC}(B_3) = \varnothing$. Hence, *read* or *write* operations in B_4 are executed at $L_{AC}(B_4) = \varnothing$ level

$$\begin{aligned} &(p = Path1) \Rightarrow prd(B_4) = B_3 \\ &\left(C_{prd(B_4)} = C_3 = \varnothing\right) \\ &\Rightarrow (L_{AC}(B_4) = L_{AC}(prd(B_4)) = L_{AC}(B_3) = \varnothing) \\ &write2(o_3) \in B_4 \Rightarrow (L_{AC}(write2) = L_{AC}(B_4) = \varnothing) \end{aligned} \tag{16}$$

According to (10), $L_{AC}(o_3) = A$. Moreover, in Table 3, the access control label of *write*2 in *Path*1 is equal to ∅. Since the *write* in the object with a greater access control label is unauthorised, *Path*1 is detected as a vulnerable path

$$\begin{aligned} &L_{AC}(o_3) = A \\ &L_{AC}(write2) = \varnothing \\ &(L_{AC}(write2) < L_{AC}(o_3)) \Rightarrow (write2(o_3) \; is \; not \; authorised) \end{aligned} \tag{17}$$

In *Path*2, since C_2 is not *null*, the access control label of B_4 is equal to A (i.e. $L_{AC}(B_4) = MAX(L_{AC}(B_2), Level(C_2)) = A$). Therefore, *read* or *write* operations in B_4 are executed at $L_{AC}(B_4) = A$ level. Moreover, $L_{AC}(o_3) = A$, and *write*2 is executed at A level, therefore *Path*2 is not vulnerable. Table 3 shows the results of labelling by the ANOVUL method in Example 1

$$\begin{aligned} &(p = Path2) \Rightarrow ((prd(B_4) = B_2) \wedge (C_2 \text{ is not satisfied})) \\ &(C_2 \text{ is not satisfied}) \Rightarrow (Level(C_2) = \neg L_{AC}(C_2) = A) \\ &\left(C_{prd(B_4)} = C_2 \neq \varnothing\right) \Rightarrow (L_{AC}(B_4) = \text{MAX}(L_{AC}(prd(B_4), \\ &Level\left(C_{prd(B_4)}\right) = \text{MAX}(L_{AC}(B_2), Level(C_2)) \\ &\text{MAX}(\varnothing, A) = A) \\ &write2(o_3) \in B_4 \Rightarrow (L_{AC}(write2) = L_{AC}(B_4) = A) \end{aligned} \tag{18}$$

In Table 3, the access control label of *write*2 in *Path*2 is equal to A. In operation *write2*, writing in data object o_3 with an access control label less than or equal to the access control label of operation *write2* is authorised

$$\begin{aligned} &L_{AC}(o_3) = A \\ &L_{AC}(write2) = A \\ &(L_{AC}(write2) \not< L_{AC}(o_3)) \Rightarrow (write2(o_3) \; is \; authorised) \end{aligned} \tag{19}$$

According to (10) and Table 3 we have the following result:

$$\begin{aligned} &(p = Path1) \Rightarrow (write2(o_3) \; is \; not \; authorised) \\ &(p = Path2) \Rightarrow (write2(o_3) \; is \; authorised) \end{aligned} \tag{20}$$

The forced browsing vulnerability in a web application occurs when an operation is performed with respect to the sequence of some pages, and the programmer controls the security check in one of the pages prior to the main page containing sensitive instructions. Therefore, an attacker can directly visit the main page and execute the sensitive operation through bypassing the security check.

In Example 2 (Fig. 8), the application is assumed to have two pages Page1 and Page2. In Page1, the user's privilege is checked to see if the user has the necessary access right to add a new user. If a user has administrator privilege, the link to Page2 is shown.

```
Page1:
  if(IsNotAdmin($_SESSION['Admin'])) //Annot-
  ation1;
    {  redirect_to_login();  } //Annotation2;
  else
    {  show_admin_pannel();  } //Annotation3;

Page2:
  get_user_input(); //Annotation4;
  if(input_is_valid()) //Annotation5;
    {  add_user()  } //Annotation6;

/*
<Annotation1>if IsNotAdmin() then Level(C2)=∅ else Level(C2)
=A </Annotation1>
<Annotation2>LAC(Login_Page)=∅</Annotation2>
<Annotation3>LAC(Admin_Page)=A</Annotation3>
<Annotation4>LAC(User_Input)=A</Annotation4>
<Annotation5>C8=∅</Annotation5>
<Annotation6>LAC(Database.User_Table)=A</Annotation6>
*/
```

Fig. 8 *Example 2: An example of forced browsing vulnerability. The annotations were added as comments to the example*

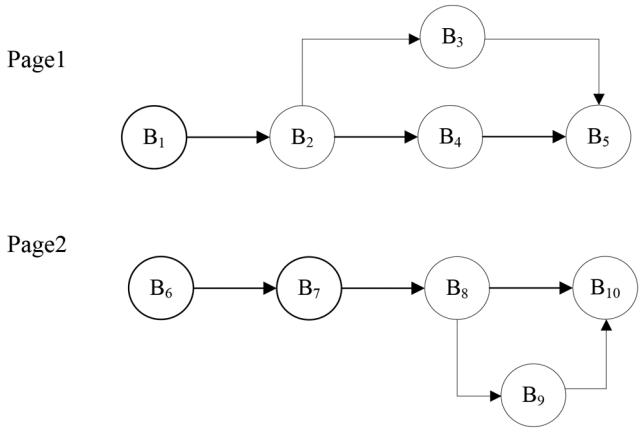

Fig. 9 *CFG for Example 2. This is an example of forced browsing vulnerability*

Therefore, a new user can be created using the add_user function in Page2.

The CFG of Example 2 can be seen in Fig. 9. In this figure, $B_1, B_6 \in B_{\text{En}}$ and $B_5, B_{10} \in B_{\text{Ex}}$. *Path*1 and *Path*2 are the possible paths in *Page*1. *Path*3 and *Path*4 are the possible paths in *Page*2:

$$\begin{aligned} Path1 &= \langle B_1, B_2, B_3, B_5 \rangle \\ Path2 &= \langle B_1, B_2, B_4, B_5 \rangle \\ Path3 &= \langle B_6, B_7, B_8, B_{10} \rangle \\ Path4 &= \langle B_6, B_7, B_8, B_9, B_{10} \rangle \end{aligned} \tag{21}$$

In the normal execution of the application, if a user does not have the necessary access right, the *login* page will appear (*Path*1). Otherwise (if the user has administrator privilege), *Path*2 is executed and the link to *Page*2 can be seen by the user. Then the user can execute *Path*4 to register a new user. In this example, the user access control in *Page*2 is performed implicitly with respect to the security check in *Page*1. This security check can be bypassed via directly visiting *Page*2. Therefore, this application has a forced browsing vulnerability.

The annotations were added to Example 2. In this example we have:

$$\begin{aligned} &o_1 = \$\_SESSION['Admin'], \quad o_2 = Login\_Page, \\ &o_3 = Admin\_Page, \quad o_4 = User\_Input, \\ &o_5 = Database\,.\,User\_Table, \\ &C_1 = C_3 = C_4 = C_5 = \varnothing, \quad C_2 = IsNotAdmin(o_1), \\ &C_6 = C_7 = C_8 = C_9 = C_{10} = \varnothing, \\ &read1 = IsNotAdmin(), \quad write1 = redirect\_to\_login(), \\ &write2 = show\_admin\_pannel(), \quad read2 = get\_user\_input(), \\ &write3 = input\_is\_valid(), \quad write4 = add\_user(), \end{aligned} \tag{22}$$

$$\begin{aligned} &B_1, B_6 \in B_{En}, \quad B_5, B_{10} \in B_{Ex}, \\ &B_2, B_3, B_4, B_7, B_8, B_9 \in B_{Bl}, \\ &B_2 = \{C_2 = read1(o_1);\ \text{if}(C_2)\}, \quad B_3 = \{write1(o_2);\}, \\ &B_4 = \{write2(o_3);\}, \quad B_7 = \{read2(o_4);\}, \\ &B_8 = \{write3(o_4);\}, \quad B_9 = \{write4(o_5);\} \end{aligned} \tag{23}$$

Considering Example 2, the following information is provided as annotations for the ANOVUL method. Basic block B_2 contains the security check *if(IsNotAdmin($_SESSION['Admin']))* such that for admin or guest user, $Level(C_2)$ is equal to A or $\varnothing$, respectively. Equation (24) shows the results of labelling based on annotations in Example 2

$$\begin{aligned} &L_{AC}(o_1) = \varnothing \\ &L_{AC}(o_2) = \varnothing \\ &L_{AC}(o_3) = \varnothing \\ &L_{AC}(o_4) = \varnothing \\ &L_{AC}(o_5) = A \\ &Level(C_2) = \begin{cases} L_{AC}(C_2) = \varnothing & if \;\; C_2 \; is \; satisfied \\ \neg L_{AC}(C_2) = A & if \;\; C_2 \; is \; not \; satisfied \end{cases} \end{aligned} \tag{24}$$

In the following, labelling is performed according to the ANOVUL method (Algorithm 2). In Fig. 9, the value of $L_{AC}(B_2)$, i.e. the access control level corresponding to B_2, is equal to $\varnothing$

$$\begin{aligned} &B_1 \in B_{En} \Rightarrow L_{AC}(B_1) = \varnothing \\ &(p = Path1) \Rightarrow prd(B_2) = B_1 \\ &\left(C_{prd(B_2)} = C_1 = \varnothing\right) \Rightarrow \\ &(L_{AC}(B_2) = L_{AC}(prd(B_2)) = L_{AC}(B_1) = \varnothing) \\ &read1(o_1) \in B_2 \Rightarrow (L_{AC}(read1) = L_{AC}(B_2) = \varnothing) \end{aligned} \tag{25}$$

According to Algorithm 3, reading object o_1 with an access control label less than or equal to the access control label of operation *read1* is authorised

$$\begin{aligned} &L_{AC}(o_1) = \varnothing \\ &L_{AC}(read1) = \varnothing \\ &(L_{AC}(read1) \not< L_{AC}(o_1)) \Rightarrow (read1(o_1) \; is \; authorised) \end{aligned} \tag{26}$$

In *Path*1, since C_2 is not *null*, $L_{AC}(B_3) = \text{MAX}(L_{AC}(B_2), Level(C_2)) = \varnothing$. Therefore, *read* or *write* operations in B_3 are executed at $L_{AC}(B_3) = \varnothing$ level

$$\begin{aligned} &(p = Path1) \Rightarrow ((prd(B_3) = B_2) \wedge (C_2 \; is \; satisfied)) \\ &(C_2 \; is \; satisfied) \Rightarrow (Level(C_2) = L_{AC}(C_2) = \varnothing) \\ &\left(C_{prd(B_3)} = C_2 \neq \varnothing\right) \Rightarrow L_{AC}(B_3) = MAX(L_{AC}(prd(B_3), \\ &Level\left(C_{prd(B_3)}\right) = MAX(L_{AC}(B_2), Level(C_2)) \\ &\text{MAX}(\varnothing, \varnothing) = \varnothing) \\ &write1(o_2) \in B_3 \Rightarrow (L_{AC}(write1) = L_{AC}(B_3) = \varnothing) \end{aligned} \tag{27}$$

Table 4 Results of labelling by the ANOVUL method in Example 2

AC labelling	*Path*1	*Path*2	*Path*3	*Path*4
operation *read*1	∅	∅	N/A	N/A
operation *read*2	N/A	N/A	∅	∅
operation *write*1	∅	N/A	N/A	N/A
operation *write*2	N/A	A	N/A	N/A
operation *write*3	N/A	N/A	∅	∅
operation *write*4	N/A	N/A	N/A	∅

The access control label of operation *write*1 is equal to ∅. In operation *write1*, writing in data object o_2 with an access control label less than or equal to the access control label of operation *write1* is authorised

$$\begin{aligned} &L_{AC}(o_2) = \varnothing \\ &L_{AC}(write1) = \varnothing \\ &(L_{AC}(write1) \not< L_{AC}(o_2)) \Rightarrow (write1(o_2)\ is\ authorised) \end{aligned} \tag{28}$$

Similarly, in *Path*2 the access control labels of B_1 and B_2 are equal to ∅. In this path, since C_2 is not *null*, $L_{AC}(B_4) = \mathrm{MAX}(L_{AC}(B_2), Level(C_2)) = A$. Therefore, *read* or *write* operations in B_4 are executed at $L_{AC}(B_4) = A$ level

$$\begin{aligned} &(p = Path2) \Rightarrow ((prd(B_4) = B_2) \wedge (C_2\ is\ not\ satisfied)) \\ &(C_2\ is\ not\ satisfied) \Rightarrow (Level(C_2) = \neg L_{AC}(C_2) = A) \\ &\left(C_{prd(B_4)} = C_2 \neq \varnothing\right) \Rightarrow (L_{AC}(B_4) = MAX(L_{AC}(prd(B_4)), \\ &Level\left(C_{prd(B_4)}\right) = MAX(L_{AC}(B_2), Level(C_2)) \\ &\mathrm{MAX}(\varnothing, A) = A) \\ &write2(o_3) \in B_4 \Rightarrow (L_{AC}(write2) = L_{AC}(B_4) = A) \end{aligned} \tag{29}$$

The access control label of operation *write*2 is equal to ∅. In operation *write2*, writing in data object o_3 with an access control label less than or equal to the access control label of operation *write2* is authorised

$$\begin{aligned} &L_{AC}(o_3) = \varnothing \\ &L_{AC}(write2) = A \\ &(L_{AC}(write2) \not< L_{AC}(o_3)) \Rightarrow (write2(o_3)\ is\ authorised) \end{aligned} \tag{30}$$

According to the ANOVUL method, the operations in basic block B_3 and B_4 are executed with the guest (∅) and administrator (A) access control levels, respectively. B_5 is regarded as the exit block of *Page*1. if the user has administrator privilege, show_admin_pannel() will be executed in B_4 to display the link to *Page2*. Therefore, *Path*2; *Path*4 is provided for the administrator to add a new user. B_6 is the entry block of *Page2*. After executing get_user_input() in B_7, the inputs required for user creation are received. B_8 contains the security check if(input_is_valid()) which does not investigate the user access control level. Therefore, C_8 is regarded as ∅. After executing add_user() in B_9, the new user will be added. B_{10} is the exit block of *Page*2. In this example, the database related object in B_9 is labelled as A (i.e. object o_5 in (24)).

In this example, $B_1, B_6 \in B_{\mathrm{En}}$. Therefore, the values of $L_{AC}(B_1)$ and $L_{AC}(B_6)$ are equal to ∅

$$\begin{aligned} &B_1 \in B_{\mathrm{En}} \Rightarrow L_{AC}(B_1) = \varnothing \\ &B_6 \in B_{\mathrm{En}} \Rightarrow L_{AC}(B_6) = \varnothing \end{aligned} \tag{31}$$

In *Path*4, since $C_6 = \varnothing$, we will have $L_{AC}(B_7) = L_{AC}(B_6) = \varnothing$. Therefore, $L_{AC}(read2) = \varnothing$

$$\begin{aligned} &(p = Path4) \Rightarrow prd(B_7) = B_6 \\ &\left(C_{prd(B_7)} = C_6 = \varnothing\right) \Rightarrow (L_{AC}(B_7) = L_{AC}(prd(B_7)) = L_{AC}(B_6) = \varnothing) \\ &read2(o_4) \in B_7 \Rightarrow (L_{AC}(read2) = L_{AC}(B_7) = \varnothing) \end{aligned} \tag{32}$$

In B_7, *read2* has access to an object with a ∅ access control label. According to Algorithm 3, reading object o_4 with an access control label less than or equal to the access control label of operation *read2* is authorised

$$\begin{aligned} &L_{AC}(o_4) = \varnothing \\ &L_{AC}(read2) = \varnothing \\ &(L_{AC}(read2) \not< L_{AC}(o_4)) \Rightarrow (read2(o_4)\ is\ authorised) \end{aligned} \tag{33}$$

According to Algorithm 2, since $C_7 = \varnothing$, the access control label of B_8 is equal to ∅. Therefore, $L_{AC}(write3) = \varnothing$

$$\begin{aligned} &(p = Path4) \Rightarrow prd(B_8) = B_7 \\ &\left(C_{prd(B_8)} = C_7 = \varnothing\right) \Rightarrow (L_{AC}(B_8) = L_{AC}(prd(B_8)) = L_{AC}(B_7) = \varnothing) \\ &write3(o_4) \in B_8 \Rightarrow (L_{AC}(write3) = L_{AC}(B_8) = \varnothing) \end{aligned} \tag{34}$$

In B_8, *read3* has access to an object with a ∅ access control label. In operation *write3*, writing in data object o_4 with an access control label less than or equal to the access control label of operation *write3* is authorised

$$\begin{aligned} &L_{AC}(o_4) = \varnothing \\ &L_{AC}(write3) = \varnothing \\ &(L_{AC}(write3) \not< L_{AC}(o_4)) \Rightarrow (write3(o_4)\ is\ authorised) \end{aligned} \tag{35}$$

In *Path*4, $C_8 = \varnothing$, therefore $L_{AC}(B_9) = L_{AC}(B_8) = \varnothing$. Hence, *read* or *write* operations in B_9 are executed at $L_{AC}(B_9) = \varnothing$ level

$$\begin{aligned} &(p = Path4) \Rightarrow prd(B_9) = B_8 \\ &\left(C_{prd(B_9)} = C_8 = \varnothing\right) \Rightarrow \\ &(L_{AC}(B_9) = L_{AC}(prd(B_9)) = L_{AC}(B_8) = \varnothing) \\ &write4(o_5) \in B_9 \Rightarrow (L_{AC}(write4) = L_{AC}(B_9) = \varnothing) \end{aligned} \tag{36}$$

According to (24), $L_{AC}(o_5) = A$. Moreover, the access control label of *write*4 is equal to ∅. Since the *write* in the object with a greater access control label is unauthorised, *Path*4 is detected as a vulnerable path

$$\begin{aligned} &L_{AC}(o_5) = A \\ &L_{AC}(write4) = \varnothing \\ &(L_{AC}(write4) < L_{AC}(o_5)) \Rightarrow (write4(o_5)\ is\ not\ authorised) \end{aligned} \tag{37}$$

Table 4 shows the results of labelling by the ANOVUL method in Example 2.

By assuming (24) as the input of ANOVUL, due to Table 4, *Path*4 is vulnerable.

The IDOR vulnerability in a web application occurs when the application is developed such that it is possible to directly access

```
Page1:
  if(selected_product()) //Annotation1;
    {  Payment(); //Annotation2;
       Download($link);  //Annotation3; }
  else
    {  Products_List();  } //Annotation4;

Page2:
  if(Payment_OK()) //Annotation5;
  {  get_user_input(); //Annotation6;
     ReadFile($link); //Annotation7;
     echo $File;  } //Annotation8;
  else
    {   redirect_to_Page1();  } //Annotation9;

/*
<Annotation1>L_AU(selected_product)=∅</Annotation1>
<Annotation2>L_AU(Payment)= ∅</Annotation2>
<Annotation3>L_AU(Download)= A</Annotation3>
<Annotation4>L_AU(Products_List)=A</Annotation4>
<Annotation5>L_AU(Payment_OK)=∅</Annotation5>
<Annotation6>L_AU(get_user_input)=∅</Annotation6>
<Annotation7>L_AU(ReadFile)= A</Annotation7>
<Annotation8>L_AU(echo)= A</Annotation8>
<Annotation9>L_AU(redirect_to_Page1)= ∅</Annotation9>
*/
```

Fig. 10 *Example 3: An example of insecure direct object reference vulnerability. The annotations were added as comments to the example*

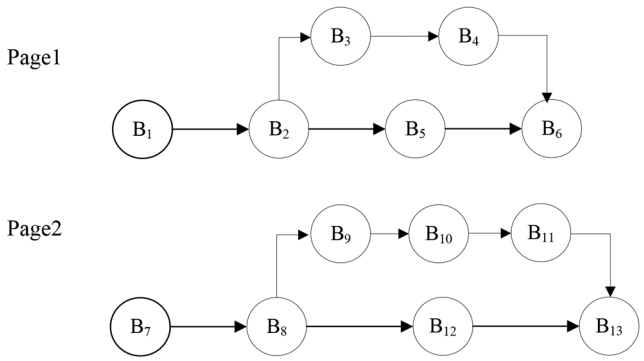

Fig. 11 *CFG for Example 3. This is an example of IDOR vulnerability*

the objects. Then an attacker can manipulate the inputs to access other objects. For example, in an electronic bookstore, if the user input is used to reference objects directly, the attacker can purchase an electronic book and manipulate the reference to retrieve any electronic book object.

Example 3 (Fig. 10) shows a sample PHP program that is vulnerable to IDOR vulnerability. If the user does not choose an electronic book, through executing the instructions in B_5, the list of books will be shown. In this example, a user first attempts to choose the product. When the payment is done the user will be automatically redirected to Page2. After verifying the payment, the link to access the electronic book will be shown. In Page2, the program provides direct access to electronic book objects based on user provided input.

In Fig. 11, the CFG of Example 3 has been shown. In this figure, $B_1, B_7 \in B_{\text{En}}$ and $B_6, B_{13} \in B_{\text{Ex}}$. *Path*1 and *Path*2 are the possible paths in *Page*1. *Path*3 and *Path*4 are the possible paths in *Page*2

$$\begin{aligned} Path1 &= \langle B_1, B_2, B_3, B_4, B_6 \rangle \\ Path2 &= \langle B_1, B_2, B_5, B_6 \rangle \\ Path3 &= \langle B_7, B_8, B_9, B_{10}, B_{11}, B_{13} \rangle \\ Path4 &= \langle B_7, B_8, B_{12}, B_{13} \rangle \end{aligned} \tag{38}$$

The annotations were added to Example 3. Therefore, in this example we have:

$$\begin{aligned} &o_1 = Database\,.\,selected\_product\_table, \\ &o_2 = Database\,.\,payment\_table, \quad o_3 = \$link, \\ &o_4 = Database\,.\,product\_table, \quad o_5 = User\_Input, \\ &o_6 = \$File, \quad o_7 = page1, \\ &C_1 = C_2 = C_3 = C_4 = C_5 = \varnothing, \\ &C_6 = C_7 = C_8 = C_9 = C_{10} = \varnothing, \\ &read1 = selected\_product(), \quad write1 = Payment(), \\ &read2 = Download(), \quad read3 = Product\_List(), \\ &read4 = Payment\_OK(), \quad read5 = get\_user\_input(), \\ &read6 = ReadFile(), \quad read7 = echo(), \\ &write2 = redirect\_to\_Page1(), \\ &B_1, B_7 \in B_{En}, \quad B_6, B_{13} \in B_{Ex}, \\ &B_2, B_3, B_4, B_5 \in B_I, \\ &B_8, B_9, B_{10}, B_{11}, B_{12} \in B_I, \\ &B_2 = \{C_2 = read1(o_1);\ \text{if}\ (C_2)\}, \quad B_3 = \{write1(o_2);\}, \end{aligned} \tag{39}$$

$$\begin{aligned} B_4 &= \{read2(o_3);\}, & B_5 &= \{read3(o_4);\}, \\ B_8 &= \{C_2 = read4(o_2);\ if(C_2)\}, & B_9 &= \{read5(o_5);\}, \\ B_{10} &= \{read6(o_3);\}, & B_{11} &= \{read7(o_6);\}, \\ B_{12} &= \{write2(o_7);\} \end{aligned} \tag{40}$$

B_2 and B_8 contain security checks. Since these checks do not check the authenticity of the data, they are not considered as data authenticity checks. Considering Example 3, the following information is provided as annotations for the ANOVUL method. Equation (41) shows the results of labelling based on annotations in Example 3

$$\begin{aligned} &L_{AU}(s_{\text{server}}) = A, \quad L_{AU}(s_{\text{client}}) = \varnothing, \\ &L_{AU}(read1) = \varnothing, \quad L_{AU}(read2) = A, \\ &L_{AU}(read3) = A, \quad L_{AU}(read4) = \varnothing, \\ &L_{AU}(read5) = \varnothing, \quad L_{AU}(read6) = A, \\ &L_{AU}(read7) = A \end{aligned} \tag{41}$$

In the following, labelling is performed according to the ANOVUL method (Algorithm 2). For each basic block b, its authenticity label is the same as the subject authenticity label

$$\begin{aligned} &L_{AU}(b) = L_{AU}(subject(b)) \\ &(subject(B_1) = s_{\text{server}}) \Rightarrow (L_{AU}(B_1) = L_{AU}(s_{\text{server}}) = A) \\ &(subject(B_1) = s_{\text{client}}) \Rightarrow (L_{AU}(B_1) = L_{AU}(s_{\text{client}}) = \varnothing) \end{aligned} \tag{42}$$

According to Algorithm 2, the authenticity label of object o_1 is equal to ∅

$$L_{AU}(o_1) = \text{MIN}(L_{AU}(B_1), L_{AU}(o_1)) = \varnothing \tag{43}$$

According to Algorithm 3, reading object o_1 with an authenticity label greater than or equal to the authenticity label of operation *read1* is authorised

$$\begin{aligned} &L_{AU}(read1) = \varnothing \\ &L_{AU}(o_1) = \varnothing \\ &(L_{AU}(o_1) \not< L_{AU}(read1)) \Rightarrow (read1(o_1)\ is\ authorised) \end{aligned} \tag{44}$$

In Example 3, object o_3 is related to a link which should not be manipulated by an unauthorised user. Therefore, the authenticity of a sensitive data object should be protected. In this example, since an unauthorised user can manipulate the reference to the electronic book object, which is a sensitive data, the program has the IDOR vulnerability.

Table 5 Evaluation results of the ANOVUL method

Application name	Files (PHP)	Known Vulnerabilities	TP[a]	FP[b]	TN[c]	Recall	Detection time
phpGradeBook 1.9.4	14	CVE-2012–1670	1	0	13	1	15:23 min
phpBMS 0.96	241	CVE-2009–3756	0	0	240	0	46:18 min
PHP iCalendar 2.24	71	CVE-2008–5840	1	1	69	1	29:08 min
phpFastNews 1.0.0	3	CVE-2008–4622	1	0	2	1	3:16 min
PhpAddEdit 1.3	57	CVE-2008–6581	1	0	56	1	26:42 min
PhpWiki 1.3.12	391	CVE-2007–3193	0	1	389	0	42:30 min
PHPDirector 0.21	111	CVE-2007–3529	0	0	110	0	29:12 min
phpGraphy 0.9.11	31	CVE-2006–1888	1	0	30	1	21:02 min
PHPMyChat 0.14.5	93	CVE-2004–2715	1	0	92	1	23:06 min
PHPSlice 0.1.4	29	CVE-2001–1367	1	0	28	1	13:53 min
OpenEMR 4	2811	CVE-2018–15152	1	1	2809	1	1:53:26 h
ukcms v1.1	601	CVE-2018–14911	1	0	600	1	40:34 min
Elefant CMS 2	633	CVE-2018–16974	0	0	632	0	47:32 min
Monstra CMS 3.0.4	508	CVE-2018–11678	1	0	507	1	31:46 min
MiniCMS 1.1	8	CVE-2018–10424, CVE-2018–10423	1	0	6	0.5	7:58 min
ZBlogPHP 1.5	428	CVE-2018–7737	1	0	427	1	35:09 min
DedeCMS 5.7	565	CVE-2018–6910	1	0	564	1	39:28 min
BlogoText 3	53	CVE-2017–17793	1	1	51	1	24:05 min
Fiyo CMS 2.0.7	352	CVE-2017–17104	1	0	351	1	25:53 min
SimpleSAMLphp 1.14.12	923	CVE-2017–12870	0	0	922	0	58:17 min
Allen Disk 1.6	62	CVE-2017–9091, CVE-2017–9090	2	0	60	1	19:28 min
GeniXCMS 1.0.2	1574	CVE-2017–8388	0	0	1573	0	1:05:38 h
BigTree CMS 4.2.17	648	CVE-2017–7881	1	1	646	1	35:24 min
Cacti 0.8.8f	234	CVE-2016–2313	1	0	233	1	28:06 min

[a]TP: True positive, [b]FP: False positive, [c]TN: True negative.

During the purchase process, *Path*1 and *Path*3 are executed. In the normal execution of the application, we have the following results:

$$\begin{aligned}(\text{subject}(B_9) = s_{\text{admin}}) &\Rightarrow (L_{AU}(B_9) = L_{AU}(s_{\text{admin}}) = A)\\ (\text{subject}(B_{10}) = s_{\text{admin}}) &\Rightarrow (L_{AU}(B_{10}) = L_{AU}(s_{\text{admin}}) = A)\end{aligned} \tag{45}$$

Pursuant to Algorithm 2, the authenticity label of object o_3 is equal to A

$$L_{AU}(o_3) = \text{MIN}(L_{AU}(B_{10}), L_{AU}(o_3)) = \text{MIN}(A, A) = A \tag{46}$$

According to (41), the authenticity label of operation *read*6 is equal to A. In basic block B_{10}, reading object o_3 with an authenticity label greater than or equal to the authenticity label of operation *read6* is authorised

$$\begin{aligned}&L_{AU}(read6) = A\\ &L_{AU}(o_3) = A\\ &(L_{AU}(o_3) \not< L_{AU}(read6)) \Rightarrow (read6(o_3)\ \textit{is authorised})\end{aligned} \tag{47}$$

If an unauthorised user manipulates the e-book link (in client side), the authenticity label of object o_3 changes to $\varnothing$

$$\begin{aligned}(\text{subject}(B_9) = s_{\text{client}}) &\Rightarrow (L_{AU}(B_9) = L_{AU}(s_{\text{client}}) = \varnothing)\\ (\text{subject}(B_{10}) = s_{\text{client}}) &\Rightarrow (L_{AU}(B_{10}) = L_{AU}(s_{\text{client}}) = \varnothing)\end{aligned} \tag{48}$$

Referring to the above results, $L_{AU}(B_{10}) = \varnothing$. Pursuant to Algorithm 2, the authenticity label of object o_3 is equal to $\varnothing$

$$L_{AU}(o_3) = \text{MIN}(L_{AU}(B_{10}), L_{AU}(o_3)) = \text{MIN}(\varnothing, A) = \varnothing \tag{49}$$

In basic block B_{10}, reading object o_3 with an authenticity label less than the authenticity label of operation *read6* is unauthorised

$$\begin{aligned}&L_{AU}(read6) = A\\ &L_{AU}(o_3) = \varnothing\\ &(L_{AU}(o_3) < L_{AU}(read6)) \Rightarrow (read6(o_3)\ \textit{is not authorised})\end{aligned} \tag{50}$$

5 Evaluation

To evaluate the ANOVUL method, a tool was implemented in C++. This tool receives an annotated source code written in PHP. Then PHP compiler [37] was used to create the CFGs of each application. Afterwards, the ANOVUL method finds the paths lacking appropriate security checks via control and data flow analysis.

To assess the ANOVUL method, we have collected a data set comprising of PHP applications with reported logic vulnerabilities that have common vulnerabilities and exposures (CVE) identifiers [38]. The proposed method is evaluated against benchmark applications and evaluation results show its effectiveness in finding logic vulnerabilities.

Table 5 shows the analysis results for the 24 web applications. Based on the results, vulnerabilities caused by bypassing one or more security checks can be detected to some extent. Table 5 shows the results of vulnerability detection by the tool, indicating a 73% true positive detection rate in the data set.

The vulnerability in phpGradeBook has allowed remote attackers to read the database via a SaveSQL action. According to Table 5, the tool has detected this vulnerability. The vulnerability in phpBMS 0.96 allows attackers to obtain confidential information via a direct request to program pages. PHC has some limitations in supporting the program statements to create CFG, and in some cases, it fails to completely create CFG. Our tool could not detect the vulnerability, as PHC was unable to completely generate CFG for the phpBMS program.

An attacker can bypass authentication and gain access to sensitive information by changing cookie values in phpFastNews, PHP iCalendar and PhpAddEdit programs. The vulnerabilities occur due to insufficient verification of user data. According to Table 5, the tool detected these vulnerabilities.

The vulnerability in PhpWiki has allowed remote attackers to bypass authentication via an empty password. Similarly, an

attacker can gain access to sensitive information via an empty value of the id parameter in PHPDirector program. These vulnerabilities were not detected, by the tool.

phpGraphy 0.9.11 allows remote attackers to bypass authentication and gain access to sensitive information via a direct request to program pages. The vulnerabilities in PHPSlice and PHPMyChat programs allow attackers to bypass authentication and gain privileges. According to Table 5, the tool could detect these vulnerabilities.

Table 5 shows the results of vulnerability detection by ANOVUL. Although most vulnerabilities could be detected by ANOVUL, it was unable to detect some vulnerabilities due to limitations in the PHC tool utilised in the implementation of our method. PHC has some limitations in supporting the program statements to generate the CFG, and in some cases, it is not able to completely generate the CFG. To overcome this shortcoming, we are attempting to create our tool and replace the utilised one.

As earlier explained that vulnerabilities are of two types: technical and logical vulnerabilities. The presented method, ANOVUL, cannot detect technical vulnerabilities like vulnerabilities inspired from improper input validation such as SQL injection and command injection vulnerabilities. However, logical vulnerabilities that arise due to control or data flow violations such as cross site request forgery, access control bypass, authentication bypass, work flow bypass, parameter manipulation and data flow violation vulnerabilities can be detected by our method. Detecting such vulnerabilities that arise due to control or data flow violations requires two important things: proper annotated source code by programmer and generation of the proper CFG.

Also, from Table 5, various application names were listed, the presented method, ANOVUL, can detect any logical vulnerabilities that arise due to control or data flow violations from those applications irrespective of application business rules. The benchmark applications with different category of functionality are used to evaluate the ANOVUL method. Table 5 illustrates true positive, false positive, and true negative rates of the ANOVUL method. According to the results, the overall precision and recall rate of ANOVUL is 80 and 73% respectively.

6 Conclusion

In this paper, we proposed a method for detecting logic vulnerabilities, which are application specific and occur in different forms depending on the application logic. ANOVUL tries to find inconsistencies between the analysis of control or data flow and annotations, which are referred to as vulnerabilities. Here, it is not required to define a pattern or rule for vulnerabilities, as there is no specific pattern for logic vulnerabilities. This is indeed the main advantage of our approach that makes it an appropriate tool for finding logic vulnerabilities.

Application security policy is required for the detection of logic vulnerabilities. In this study on the ANOVUL method, security policy was annotated with source code. Then a static analysis was conducted to detect application vulnerabilities. This method was implemented and executed on the vulnerable versions of some applications. The results got to indicate the success of this vulnerability detection method.

In this work, we assumed that there are two levels of security labels: *administrator* and *guest*. In future work, we plan to extend the method proposed here to have more levels of security labels by using lattice-based access control models.

7 References

[1] Sommerville, I.: '*Software Engineering, 9th edition*' (Addison-Wesley, Boston, 2010)

[2] Eckerson, W.W.: 'Three tier client/server architectures: achieving scalability, performance, and efficiency in client/server applications', *Open Inf. Syst.*, 1995, **3**, (20), pp. 46–50

[3] 'Understanding the Differences Between Technical and Logical Web Application Vulnerabilities', Available at. https://www.netsparker.com/blog/web-security/logical-vs-technical-web-application-vulnerabilities/

[4] 'Logical and Technical Vulnerabilities – What they are and how can they be detected?', Available at. https://www.acunetix.com/blog/web-security-zone/logical-and-technical-vulnerabilities/

[5] Li, X., Xue, Y.: 'A survey on server-side approaches to securing web applications', *ACM Comput. Surv. CSUR*, 2014, **46**, (4), p. 54

[6] Ghorbamzadeh, M., Shahriari, H.R.: 'Static detection of second order injection vulnerabilities using query dependency graph'. (in Persian) presented at the 17th CSI Computer Conf., Tehran, 2012, pp. 579–584

[7] Felmetsger, V., Cavedon, L., Kruegel, C., *et al.*: 'Toward automated detection of logic vulnerabilities in web applications'. USENIX Security Symp., Washington, DC, USA, 2010, pp. 143–160

[8] Sun, F., Xu, L., Su, Z.: 'Detecting logic vulnerabilities in e-commerce applications'. Presented at the Network and Distributed System Security (NDSS) Symp., San Diego, CA, USA, 2014

[9] Doupe, A., Cova, M., Vigna, G.: 'Why Johnny can't pentest: an analysis of black-box web vulnerability scanners'. DIMVA'10: Proc. of the Conf. on Detection of Intrusions and Malware and Vulnerability Assessment, Bonn, Germany, 2010

[10] Balzarotti, D., Cova, M., Felmetsger, V.V., *et al.*: 'Multi-module vulnerability analysis of web-based applications'. Proc. of the 14th ACM Conf. on Computer and communications security, Alexandria, VA, USA, 2007, pp. 25–35

[11] Wang, R., Chen, S., Wang, X., *et al.*: 'How to shop for free online – security analysis of cashier-as-a service based web stores'. Proc. of Symp. on Security and Privacy, Oakland, CA, USA, 2011

[12] Wang, R., Chen, S., Wang, X.: 'Signing me onto your accounts through Facebook and Google: a traffic-guided security study of commercially deployed single-sign-on web services'. Proc. of Symp. on Security and Privacy, San Francisco, CA, USA, 2012

[13] Xing, L., Chen, Y., Wang, X., *et al.*: 'Integuard: toward automatic protection of third-party web service integrations'. Proc. of Network and Distributed System Security, San Diego, CA, USA, 2013

[14] Shahriar, H., Zulkernine, M.: 'Mitigating program security vulnerabilities: approaches and challenges', *ACM Comput. Surv.*, 2012, **44**, (3), p. 11

[15] Ghaffarian, S.M., Shahriari, H.R.: 'Software vulnerability analysis and discovery using machine-learning and data-mining techniques: a survey', *ACM Comput. Surv.*, 2017, **50**, (4), pp. 56:1–56:36

[16] Cova, M., Balzarotti, D., Felmetsger, V., *et al.*: 'Swaddler: an approach for the anomaly-based detection of state violations in web applications'. Recent Advances in Intrusion Detection, Gold Coast, Australia, 2007, pp. 63–86

[17] Pellegrino, G., Balzarotti, D.: 'Toward black-box detection of logic flaws in web applications'. Presented at the Network and Distributed System Security (NDSS) Symp., San Diego, CA, USA, 2014

[18] Bisht, P., Hinrichs, T., Skrupsky, N., *et al.*: 'WAPTEC: whitebox analysis of web applications for parameter tampering exploit construction'. Proc. of the 18th ACM Conf. on Computer and communications security, Chicago, IL, USA, 2011, pp. 575–586

[19] Bisht, P., Hinrichs, T., Skrupsky, N., *et al.*: 'Automated detection of parameter tampering opportunities and vulnerabilities in web applications', *J. Comput. Secur.*, 2014, **22**, (3), pp. 415–465

[20] 'Category:OWASP Top Ten Project – OWASP', Available at. https://www.owasp.org/index.php/Category: OWASP_Top_Ten_Project

[21] Weinberger, J.H.W.: 'Analysis and Enforcement of Web Application Security Policies', PhD. dissertation, Computer Science Dept., University of California, Berkeley, 2012

[22] Homaei, H., Shahriari, H.R.: 'Seven years of software vulnerabilities: the ebb and flow', *IEEE Secur. Priv.*, 2017, **15**, (1), pp. 58–65

[23] Chaudhuri, A., Foster, J.S.: 'Symbolic security analysis of Ruby-on-rails web applications'. Proc. of the 17th ACM Conf. on Computer and communications security, Chicago, IL, USA, 2010, pp. 585–594

[24] Weaver, J.B.P.M.M., Evans, M.Z.D.: 'Guardrails: A data-centric web application security framework'. 2nd USENIX Conf. on Web Application Development, Portland, OR, USA, 2011, p. 1

[25] Zhu, J., Chu, B., Lipford, H., *et al.*: 'Mitigating access control vulnerabilities through interactive static analysis'. Proc. of the 20th ACM Symp. on Access Control Models and Technologies, New York, NY, USA, 2015, pp. 199–209

[26] Sun, F., Xu, L., Su, Z.: 'Static detection of access control vulnerabilities in web applications'. USENIX Security Symp., San Francisco, CA, USA, 2011, Vol. 64

[27] Alkhalaf, M., Choudhary, S.R., Fazzini, M., *et al.*: 'Viewpoints: differential string analysis for discovering client-and server-side input validation inconsistencies'. Proc. of the 2012 Int. Symp. on Software Testing and Analysis. ACM, Minneapolis, MN, USA, 2012

[28] Bisht, P., Hinrichs, T., Skrupsky, N., *et al.*: 'No tamper: automatic blackbox detection of parameter tampering opportunities in web applications'. Proc. of the 17th ACM Conf. on Computer and communications security, Chicago, IL, USA, 2010, pp. 607–618

[29] Li, X., Xue, Y.: 'Logicscope: automatic discovery of logic vulnerabilities within web applications'. Proc. of the 8th ACM SIGSAC symp. on Information, Computer and Communications Security, Berlin, Germany, 2013, pp. 481–486

[30] Son, S., McKinley, K.S., Shmatikov, V.: 'Fix Me Up: repairing access-control bugs in web applications'. NDSS, San Diego, CA, USA, 2013

[31] Monshizadeh, M., Prasad, N., Venkatakrishnan, V.N.: 'MACE: detecting privilege escalation vulnerabilities in web applications'. Proc. of the 2014 ACM SIGSAC Conf. on Computer and Communications Security, 2014

[32] Skrupsky, N., Bisht, P., Hinrichs, T., *et al.*: 'Tamperproof: a server-agnostic defense for parameter tampering attacks on web applications'. Proc. of the third ACM Conf. on Data and application security and privacy, ACM, San Antonio, TX, USA, 2013

[33] Rocchetto, M., Ochoa, M., Dashti, M.T.: 'Model-based detection of CSRF'. IFIP Int. Information Security Conf., Marrakech, Morocco, 2014

[34] Willem, D.G., Devriese, D., Nikiforakis, N., *et al.*: 'Secure multi-execution of web scripts: theory and practice', *J. Comput. Secur.*, 2014, **22**, pp. 469–509

[35] Akhawe, D., Barth, A., Lam, P.E., *et al.*: 'Towards a formal foundation of web security'. 2010 23rd IEEE Computer Security Foundations Symp., Edinburgh, UK, 2010, pp. 290–304
[36] Hedin, D., Bello, L., Sabelfeld, A.: 'Information-flow security for JavaScript and its APIs', *J. Comput. Secur.*, 2016, **24**, pp. 181–234
[37] 'PHC – The Open Source PHP Compiler', Available at. http://www.phpcompiler.org/
[38] 'CVE security vulnerability database. Security vulnerabilities, exploits, references and more', Available at. http://www.cvedetails.com/